

Отчет к лабораторной работе

Лабораторная работа

“Численное решение обыкновенных дифференциальных уравнений”

Шуман Владислав Олегович

3 курс 4 группа

вариант 9-11

корневая директория с лабораторной работой Github

[Github](#)

в ней лежат

- исходный код в файле irupb
- /отчет
- /условие (pdf или pb) и /теория (вырванные из контекста главы конспектов)

Дата последних изменений: 24 мая 2026

Условие

Задание

1. Дана задача Коши для ОДУ. Решить эту задачу на данном интервале $[x_0, X]$ аналитически или численно (самостоятельно или с помощью *Mathematica* или любой другой программы). Разрешается вместо задачи из варианта взять любую задачу небесной или земной механики.
2. Построить график полученного решения, вычислить значение решения в точке X . Если дана система ОДУ, необходимо построить графики каждой компоненты решения.
3. Написать программу, которая решает данную задачу Коши соответствующим вашему варианту численным методом с адаптивным выбором шага по правилу Рунге (см. конспект). Если метод многошаговый – используйте фиксированный шаг, стартовые значения следует брать из решения, полученного в пункте 1. Если ОДУ имеет порядок выше первого, его необходимо предварительно свести к системе первого порядка путем введения вспомогательных переменных.
4. Построить точечные графики численного решения, полученного количестве шагов около $N=200$, совместить их с графиками из пункта 2.
5. Построить логарифмическую диаграмму сходимости. Для многошаговых методов с постоянным шагом по оси абсцисс на диаграмме откладывается величина шага h , соответствующая количеству отрезков $N = 10^k$, $k = 1, 2, \dots, 6$. Для одношаговых методов с адаптивной сеткой по оси абсцисс откладывается требуемая величина локальной погрешности $\varepsilon = 10^{-k}$, $k = 1, 2, \dots, 10$. По оси ординат откладывается погрешность $r_k = |y(X) - y_N|$. Здесь $y(X)$ – точное решение, вычисленное в пункте 2, y_N – приближенное решение, полученное вашей программой.
6. Составить отчет, который содержит
 - » Описание решения задачи Коши из пункта 1. Если используются сторонние программы – привести команды, с помощью которых получено решение.
 - » Сведение к системе первого порядка (если решается уравнение порядка выше 1)
 - » Совмещенные графики точного и приближенного решений из пункта 4
 - » Диаграмма из пункта 5
 - » Код программы

Метод: 9

Задача: 11

9. Двухшаговый неявный метод Адамса

11. $x'(t) = -3(x(t) - y(t))$, $y'(t) = x(t)(-z(t)) + 26.5x(t) - y(t)$, $z'(t) = x(t)y(t) - z(t)$, $x(0) = z(0) = 0$, $y(0) = 1$, $[x_0, X] = [0, 10]$

1. Дана задача Коши для ОДУ. Решить эту задачу на данном интервале $[x_0, X]$ аналитически или численно (самостоятельно или с помощью *Mathematica* или любой другой программы). Разрешается вместо задачи из варианта взять любую задачу небесной или земной механики.

- Описание решения задачи Коши из пункта 1. Если используются сторонние программы – привести команды, с помощью которых получено решение.

$$[t_0, T] = [0, 10]$$

$$\begin{cases} x'(t) = -3(x(t) - y(t)) \\ y'(t) = x(t)(-z(t)) + 26.5x(t) - y(t) \\ z'(t) = x(t)y(t) - z(t) \end{cases}$$

$$x(0)=0, y(0)=1, z(0)=0$$

$$T=10$$

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# правая часть системы
def function(t, u):
    x, y, z = u
    dxdt = -3 * (x - y)
    dydt = x * (-z) + 26.5 * x - y
    dzdt = x * y - z
    return np.array([dxdt, dydt, dzdt])

# решение системы на интервале
sol = solve_ivp(
    function, # наша правая часть
    [0, 10], # интервал
    [0, 1, 0], # начальные условия
    rtol=1e-12, # относительная
    atol=1e-12 # абсолютная погрешности
)
```

```
# значение в точке 10
x_end, y_end, z_end = sol.y[:, -1]
```

2. Построить график полученного решения, вычислить значение решения в точке X . Если дана система ОДУ, необходимо построить графики каждой компоненты решения.

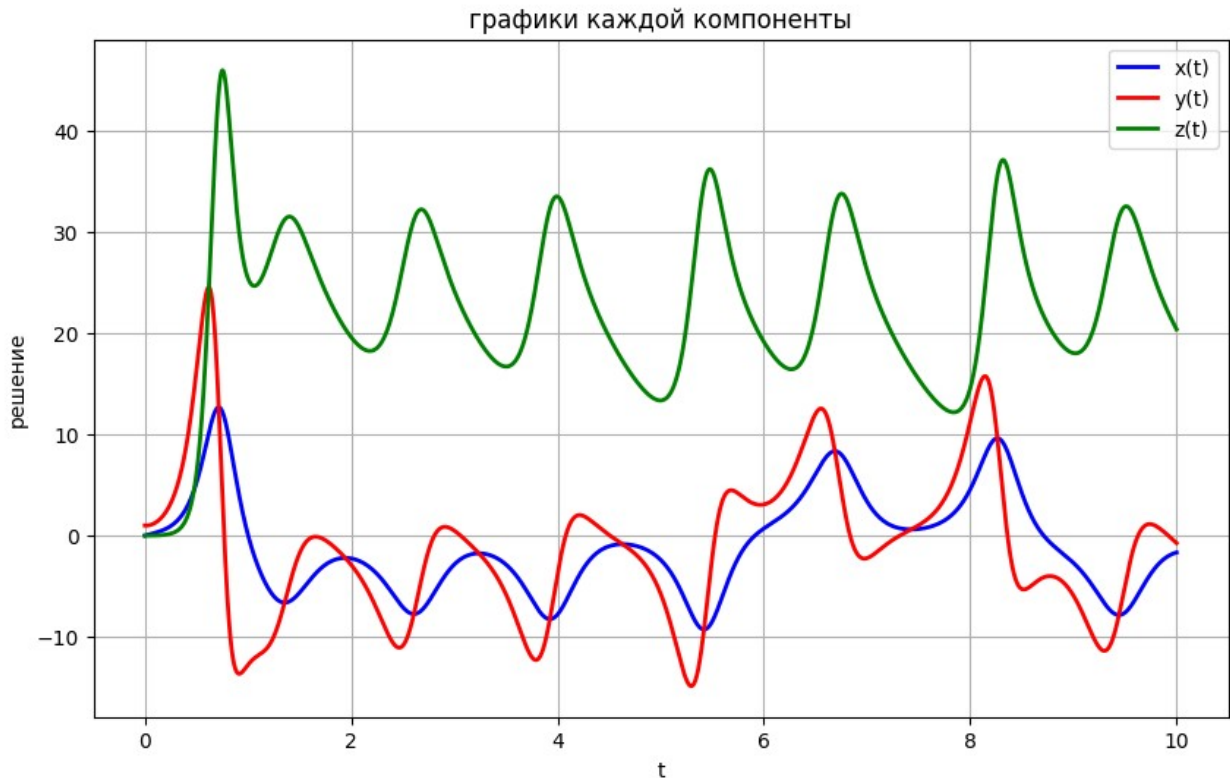
```
print(f"x(10) = {x_end:.12f}")
print(f"y(10) = {y_end:.12f}")
print(f"z(10) = {z_end:.12f}")

plt.figure(figsize=(10, 6))
plt.plot(sol.t, sol.y[0], label="x(t)", color="blue", linewidth=2)
plt.plot(sol.t, sol.y[1], label="y(t)", color="red", linewidth=2)
plt.plot(sol.t, sol.y[2], label="z(t)", color="green", linewidth=2)
plt.xlabel("t")
plt.ylabel("решение")
plt.title("графики каждой компоненты")
plt.legend()
plt.grid(True)
plt.show()
```

$x(10) = -1.665775403729$

$y(10) = -0.734529120469$

$z(10) = 20.378666398905$



- 3.** Написать программу, которая решает данную задачу Коши соответствующим вашему варианту численным методом с адаптивным выбором шага по правилу Рунге (см. конспект). Если метод многошаговый – используйте фиксированный шаг, стартовые значения следует брать из решения, полученного в пункте 1. Если ОДУ имеет порядок выше первого, его необходимо предварительно свести к системе первого порядка путем введения вспомогательных переменных.

Мой метод

9. Двухшаговый неявный метод Адамса

фрагменты теории, на которые я хочу сослаться

Подставляя всё это в (27.3), получаем общий вид многошаговых методов квадратурного типа:

$$y_k = y_{k-\ell} + \sum_{i \in J} f_i \int_{x_{k-\ell}}^{x_k} \Lambda_i(x) dx. \quad (27.7)$$

Таким образом, для однозначного определения метода нужно задать параметры ℓ и J . Заметим также, что построенные методы будут неявными в случае $k \in J$ (f_k зависит от y_k) и явными во всех остальных случаях.

27.2.3 Неявные методы Адамса

Неявные методы Адамса получаются при

$$\ell = 1 \quad \text{и} \quad J = \{0, 1, \dots, k\}.$$

В точности повторяя предыдущие рассуждения, получаем общий вид этих методов:

$$y_k = y_{k-1} + h \sum_{i=0}^k \beta_i f_i, \quad (27.12)$$

где

$$\beta_i = \frac{(-1)^{k-i}}{i!(k-i)!} \int_{k-1}^k \prod_{\substack{j=0 \\ j \neq i}}^k (t-j) dt. \quad (27.13)$$

Соответствующая таблица значений приведена ниже.

k	β_0	β_1	β_2	β_3
1	$\frac{1}{2}$	$\frac{1}{2}$	$\times$	$\times$
2	$-\frac{1}{12}$	$\frac{8}{12}$	$\frac{5}{12}$	$\times$
3	$\frac{1}{24}$	$-\frac{5}{24}$	$\frac{19}{24}$	$\frac{9}{24}$

$$x_i = x_0 + ih.$$

27.4 Важное замечание о форме записи методов

Рассмотрим, к примеру, явный метод Адамса (27.10):

$$y_k = y_{k-1} + h \sum_{i=0}^{k-1} \beta_i f_i.$$

Такая форма представления удобна при выводе формул и при анализе методов. Но для практической реализации удобнее переписать метод как правило вычисления y_{n+1} в виде (27.1), то есть в нашем случае как

$$y_{n+1} = y_n + h \sum_{i=0}^{k-1} \beta_{k-i} f_{n-i}.$$

q от 1 до $k+1$. Следовательно, неявный k -шаговый метод Адамса имеет порядок точности не менее $k+1$. Не очень трудно показать, что более высокого порядка достичь нельзя.

Замечание 27.1. При реализации неявных многошаговых методов требуется решить систему из $N = n$ нелинейных уравнений (n — размерность интегрируемой системы ОДУ). Это выгодно отличает их от неявных методов РК, для которых, как мы помним, $N = sn$.

$$\vec{u}(t) = \begin{pmatrix} x(t) \\ y(t) \\ z(t) \end{pmatrix}$$

```
# u0 и u1 получить
def get_starting_values(step_size):
    u0 = np.array([0.0, 1.0, 0.0])
    ref_solve = solve_ivp(
        function,
        [0, step_size],
        u0,
        t_eval=[step_size],
        rtol=1e-12,
        atol=1e-12
    )
    u1 = ref_solve.y[:, 0].flatten()
    return u0, u1

from scipy.optimize import fsolve # для решения НСУ
```

```

# функция метода
def implicit_adams_2step(step_size, num_steps):
    t_values = np.linspace(0, 10, num_steps + 1) # разбит [0,10] на
num_steps + 1
    u_values = np.zeros((num_steps + 1, 3)) # u0, u1, ... u_{n+1}

    # заносим начальные
    u_values[0], u_values[1] = get_starting_values(step_size)

    # правые части
    f_values = np.zeros((num_steps + 1, 3)) # f0, f1, f2, ...
    f_values[0] = function(t_values[0], u_values[0])
    f_values[1] = function(t_values[1], u_values[1])

    for n in range(1, num_steps):
        # НСУ чтобы найти u_{n+1}
        def equations_system(u_n_plus_one):
            f_n_plus_one = function(t_values[n+1], u_n_plus_one)
            # разность Л и П из (27.12)
            return u_n_plus_one - u_values[n] - (step_size / 12.0) *
(5 * f_n_plus_one + 8 * f_values[n] - f_values[n-1])

            u_values[n+1] = fsolve(equations_system, u_values[n]) # u_{n+1}
            f_values[n+1] = function(t_values[n+1], u_values[n+1]) # f_{n+1}

    return t_values, u_values

```

- 4. Построить точечные графики численного решения, полученного количестве шагов около $N=200$, совместить их с графиками из пункта 2.**

```

# константы
num_steps_plot = 200
step_size_plot = 10.0 / num_steps_plot

# численное решение нашим методом
t_adams, u_adams = implicit_adams_2step(step_size_plot,
num_steps_plot)

# построение совмещенных графиков
plt.figure(figsize=(12, 8))

# пункт 2
plt.plot(sol.t, sol.y[0], label="x(t) пункт 2", color="blue",
linewidth=1.5, alpha=0.6)
plt.plot(sol.t, sol.y[1], label="y(t) пункт 2", color="red",

```

```

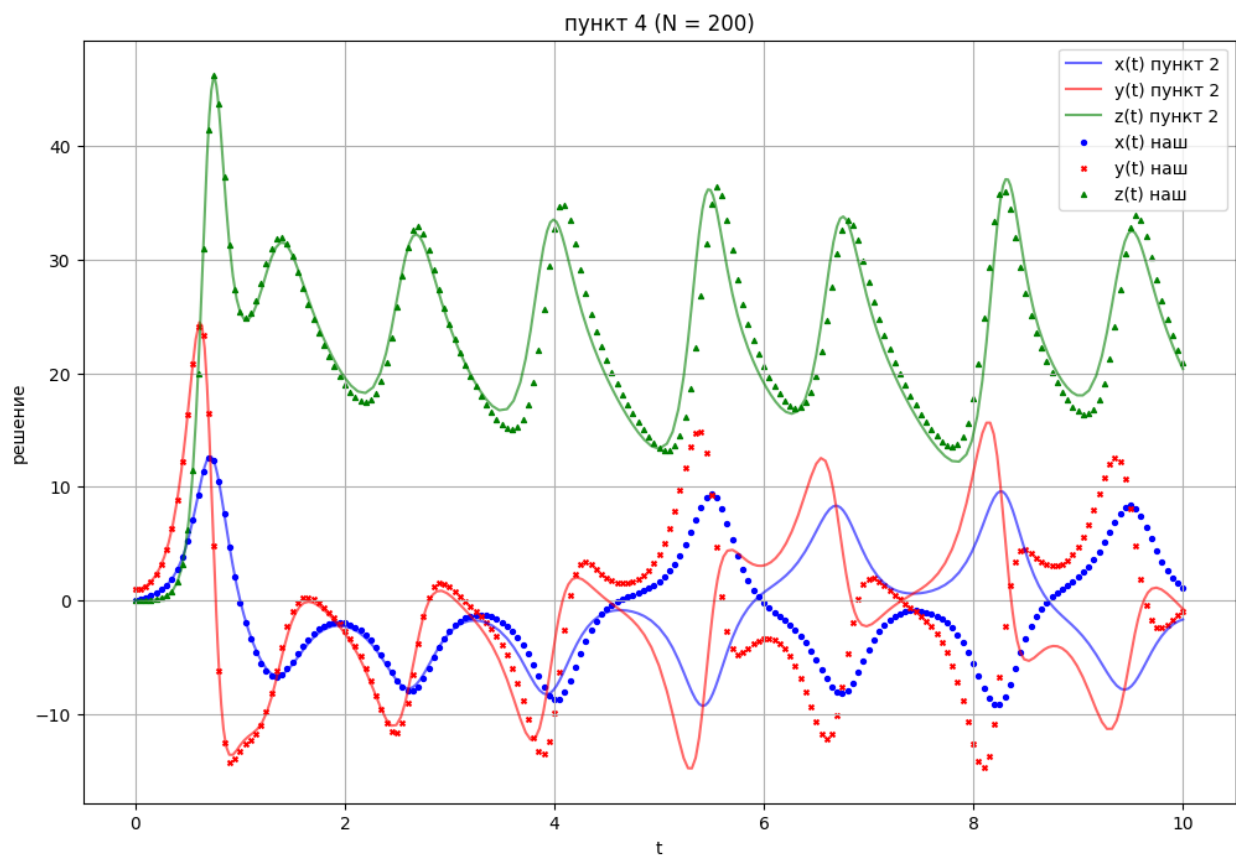
linewidth=1.5, alpha=0.6)
plt.plot(sol.t, sol.y[2], label="z(t) пункт 2", color="green",
linewidth=1.5, alpha=0.6)

# точки нашего метода
plt.scatter(t_adams, u_adams[:, 0], label="x(t) наш", color="blue",
marker="o", s=8)
plt.scatter(t_adams, u_adams[:, 1], label="y(t) наш", color="red",
marker="x", s=8)
plt.scatter(t_adams, u_adams[:, 2], label="z(t) наш", color="green",
marker="^", s=8)

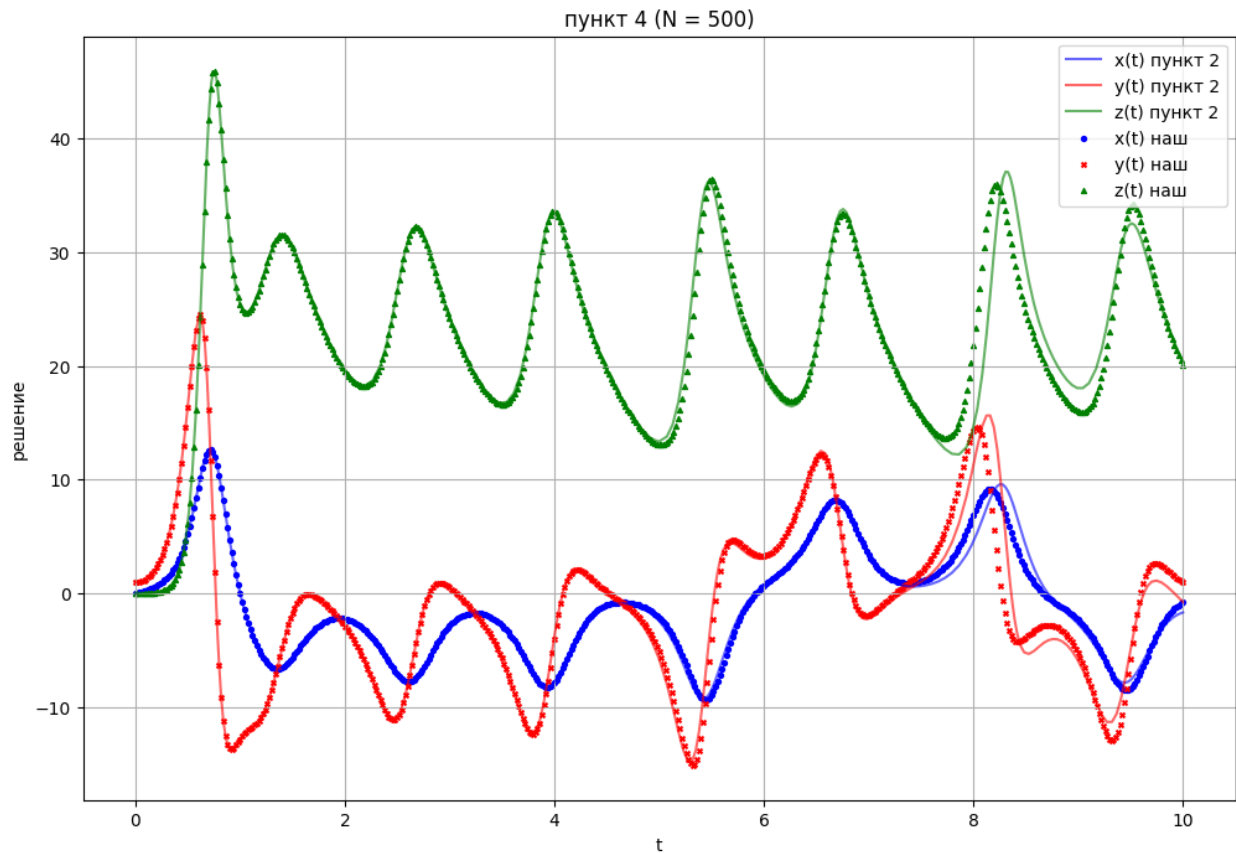
# оформление графика
plt.xlabel("t")
plt.ylabel("решение")
plt.title(f"пункт 4 (N = {num_steps_plot})")
plt.legend()
plt.grid(True)
plt.show()

```

как видно из графика,



точки хорошо ложатся на график только до $t=4$, а далее начинает симметрично отражаться приблизительно относительно $x=0$. при чем -- при $N=500$ все намного лучше



с чем это связано?

- методы Адамса нуль-устойчивы

Многошаговый метод является нуль-устойчивым если и только если все корни многочлена $\rho(\xi) = \sum_{j=0}^k \alpha_j \xi^j$ принадлежат единичному кругу комплексной плоскости, причём на границе этого круга нет кратных корней.

Нетрудно показать, что все методы Адамса устойчивы. Трудно показать, что методы ФДН устойчивы при $k \leq 6$ и неустойчивы при всех остальных k .

- глобальной ошибкой и как h влияет на это

26.1 Глобальная погрешность одношаговых методов

Глобальной погрешностью одношагового метода решения задачи Коши называется погрешность, накопленная после нескольких последовательных шагов.

Рассмотрим пошаговый процесс численного интегрирования задачи (22.1):

$$y_{k+1} = \Phi(x_k, y_k, x_{k+1}), \quad k = 0, 1, \dots, N-1.$$

Обозначим $x_N = x_0 + H = X$. Глобальной погрешностью метода Φ называют величину

$$y(X) - y_N.$$

Нас интересует зависимость величины глобальной погрешности от частоты сетки $\{x_k\}$. Для этого удобно рассмотреть равномерную сетку

$$\{x_k = x_0 + kh\}_{k=0}^N, \quad h = H/N.$$

Тогда величина h является естественным рычагом управления точностью, поэтому рассмотрим зависимость глобальной погрешности от h . Приближённое решение $y_N \approx y(X)$ обозначим $\tilde{y}(X, h)$ и, не вдаваясь в технические детали, приведём следующий важный результат.

- если подставить $-x$ и $-y$ вместо x и y , то получим симметрию: $(x, y, z) \rightarrow (-x, -y, z)$, то есть -- тоже является решением

5. Построить логарифмическую диаграмму сходимости. Для многошаговых методов с постоянным шагом по оси абсцисс на диаграмме откладывается величина шага h , соответствующая количеству отрезков $N = 10^k$, $k = 1, 2, \dots, 6$. Для одношаговых методов с адаптивной сеткой по оси абсцисс откладывается требуемая величина локальной погрешности $\varepsilon = 10^{-k}$, $k = 1, 2, \dots, 10$. По оси ординат откладывается погрешность $r_N = |y(X) - y_N|$. Здесь $y(X)$ – точное решение, вычисленное в пункте 2, y_N – приближенное решение, полученное вашей программой.

```
# эталон
u_exact_end = np.array([x_end, y_end, z_end])
```

```

# для сохранения шагов и погрешностей
step_sizes = []
errors = []

# число шагов  $N = 10^k$ 
k_values = [1, 2, 3, 4, 5, 6]

for k in k_values:
    num_steps_calc = 10**k
    step_size_calc = 10.0 / num_steps_calc

    # решаем
    _, u_calc = implicit_adams_2step(step_size_calc, num_steps_calc)
    u_calc_end = u_calc[-1]

    # погрешность в конечной точке
    error_norm = np.linalg.norm(u_exact_end - u_calc_end)

    step_sizes.append(step_size_calc)
    errors.append(error_norm)

    print(f"N = {num_steps_calc:7d} | h = {step_size_calc:8.6f} | err
= {error_norm:.2e}")

# построение логарифмической диаграммы сходимости
plt.figure(figsize=(10, 6))
plt.loglog(step_sizes, errors, "o-", color="purple", linewidth=2,
label="погрешность")

#  $O(h^3)$ 
ref_slope = [h**3 * (errors[-1] / (step_sizes[-1]**3)) for h in
step_sizes]
plt.loglog(step_sizes, ref_slope, "--", color="gray", label="O(h^3)")

plt.xlabel("величина шага h")
plt.ylabel("погрешность rN")
plt.title("логарифмическая диаграмма сходимости")
plt.legend()
plt.grid(True, which="both", ls="--")
plt.show()

```

N = 10 | h = 1.000000 | err = 2.71e+02

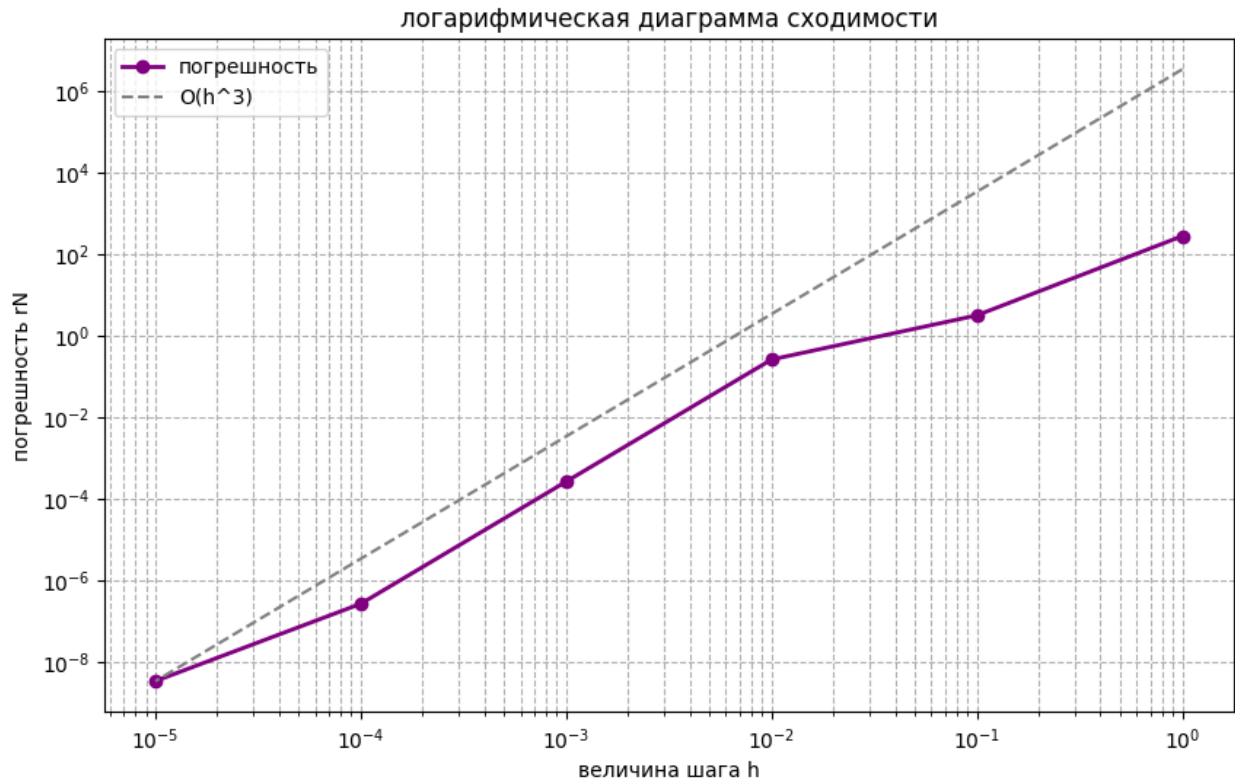
N = 100 | h = 0.100000 | err = 3.09e+00

N = 1000 | h = 0.010000 | err = 2.55e-01

N = 10000 | h = 0.001000 | err = 2.59e-04

N = 100000 | h = 0.000100 | err = 2.62e-07

$N = 1000000$ | $h = 0.000010$ | $err = 3.32e-09$



исходя из графика видно

- чем меньше шаг (то есть -- чем больше N), тем меньше погрешность
- порядок точности метода соответствует/близок к теоретическому (2-шаговый, то есть $p = 2 + 1 = 3$)